

최종

▼ 만든 파일 설명

① 원래 파이프라인 — 오늘 100건 샘플링부터 빈 텍스트 발견 전까지

- `scripts/sample_v4_100.py` — `hk_electrical_recent_full.csv` (16,964행)에서 100건 샘플링(무작위50 + 검증용 50: 다중CMPLID/UNKNOWN+형제있음/UNKNOWN+형제없음)
- `data/samples/sample_100_v4.csv` — 위 샘플링 결과 100행(51개 원본 컬럼 + sample_group)
- `docs/struct_prompt_v4.md` (원본, 예시 6개) — v4 구조화 프롬프트
- `scripts/str01_batch_structurize.py` (v4 지원 추가) — CMPLID 단위 처리, COMPDESC 앵커링, 형제 COMPDESC 조희
- `data/processed/str01_sample100_v4_results.jsonl` (1차) — 100건 구조화 결과, insufficient_info=true 14건이 빈 배열/빈 문자열로 나왔던 버전
- `data/processed/str01_sample100_v4_report.md` — 위 결과 리포트(STR-01/02, 스키마위반0·인용불일치0)
- `scripts/struct_verify_v4.py` (원본) — v4 독립 재검증 스크립트
- `scripts/str03_consistency_analyze_v4.py` (원본) — STR-03 자기일관성 분석 스크립트
- `data/processed/str03_v4_run1.jsonl` / `run2.jsonl` / `run3.jsonl` — temperature=0.4 3회 독립 실행 결과
- `data/processed/str03_consistency_v4_report.md` / `str03_consistency_v4_detail.csv` — STR-03 분석 리포트(안정74/부분흔들림16/불안정10)
- `scripts/grading_sheet_v4.py` — 수동 채점표 생성 스크립트
- `data/samples/grading_sheet_v4.csv` — 채점표(이 시점엔 빈 값 있는 채로 생성됨)

② 지금 새로 고친 수정판 (예시4 삭제 + 코드 후처리 반영 이후)

- `docs/struct_prompt_v4.md` (수정) — "진짜 정보부족" 예시(구 예시4) 삭제, 예시 5개로 재정렬
- `scripts/str01_batch_structurize.py` (수정) — `symptoms` / `evidence_quote` 빈 값이면 코드가 `["정보 부족으로 판단 불가"]` / `["(정보 부족으로 근거 문장 없음)"]` 로 강제 채우는 후처리 추가
- `scripts/struct_verify_v4.py` (수정) — 원본 CDESCR 모지바케 복구(`fix_mojibake`) 누락 버그 수정 + 안내문 구 placeholder 인식 추가
- `data/processed/str01_sample100_v4_results.jsonl` (같은 파일명, 내용 갱신) — 위 수정판으로 100건 재실행, 빈 값 0건 확인
- `data/processed/str01_sample100_v4_fixed_report.md` (신규 파일명) — 수정판 검증 리포트
- `data/samples/grading_sheet_v4.csv` (같은 파일명, 내용 갱신) — 수정판 결과로 채점표 재생성, 빈 칸 0건
- `scripts/str03_consistency_analyze_v4.py` (수정) — 입출력 경로를 `str03_v4_fixed_*` 로 변경
- `data/processed/str03_v4_fixed_run1.jsonl` (완료, 100건) / `run2.jsonl` (진행 중, 38/100) / `run3.jsonl` (대기) — temperature=0.4 3회 재실행
- `data/processed/str03_consistency_v4_fixed_report.md` / `_detail.csv` — 아직 생성 전(3회 실행 끝나야 만들어 짐)

STR-01 — 배치 구조화 API

▼ 1트 때 발견된 문제 → 개선

1. 이런식으로 meeting 파일로부터 제미나이가 잘못 매핑하게 함

실제 대분류가 이미 raw data에 있는데 이걸 무시하고, 불만 텍스트만 읽어서 part_category를 스스로 분류하고 판단하게함

→ 그래서 이미 raw data에 존재하는 COMPDESC를 사용

이 7개와 part_category 8개를 대응시켜보면

COMPDESC 대분류(7개)	→	part_category(8개)
ELECTRICAL SYSTEM (6,837)	분화됨	ELECTRICAL_SYSTEM, INSTRUMENT_CLUSTER, PROPULSION_BATTERY, BRAKES_ELECTRONIC, POWERTRAIN_SW, NON_ELECTRICAL — 하나가 6개로 쪼개짐
FORWARD COLLISION AVOIDANCE (2,057)	합쳐짐	ADAS
VEHICLE SPEED CONTROL (1,439)	합쳐짐	ADAS
LANE DEPARTURE (630)	합쳐짐	ADAS
BACK OVER PREVENTION (273)	합쳐짐	ADAS
ELECTRONIC STABILITY CONTROL (17)	합쳐짐	ADAS
UNKNOWN OR OTHER (5,711)	원문 다시 읽어서 재 판정	8개 중 아무거나 될 수 있음

2. 하나의 odino 안에서도 서로 다른 유형의 불만이 담긴 경우는, 분류가 애매하다(그래서 정확도도 낮았음)

sample_100_for_meeting.csv 파일에는 complid도 없고, 16900건에서 100건을 샘플링 해올 때 같은 이미 odino 만으로도 다 구분가능해서 오해가 생김(높은 확률로 겹치는게 없이 샘플링되었었던 것). 근데 실제로는 하나의 odino에서도 compdesc에 따라서 다른 행의 데이터로 존재하고 그건 complid값으로 구분 가능하다.

→ 애초에 Raw data에서 불러올 때, COMPLID, ODINO는 그대로 가져온다

→ part_category라는 잘못된 컬럼을 제거하고 기존 raw data의 COMPDESC를 그대로 사용한다, 그러나 세분화를 최대치로 시키면 4단계까지 분리된다, 그래서 대분류(COMPDESC1), 중분류(COMPDESC2)로만 나누어도 98.7% 커버 가능하다.

→ symptoms, severity, driving_context, evidence_quote은 1트 때도 불만 텍스트 원문을 보기는 했으나, 여기서는 complid로 같은 odino에서도 분류된 불만유형을 보게 한다.

핵심 차이는 “무엇을 식별하는 단위인가”예요.

	CMPLID	ODINO
식별 단위	행(row) 하나	신고(complaint) 하나
같은 신고 안 여러 부품 태그 시	매번 다른 값 (행마다 고유)	똑같은 값 (신고가 같으니까)
비유	접수증 날장 번호	사건 파일 번호

실제로 확인했던 예시로 보면 (ODINO 11542050)

CMPLID=1924163 | ODINO=11542050 | COMPDESC=ELECTRICAL SYSTEM
 CMPLID=1924165 | ODINO=11542050 | COMPDESC=FORWARD COLLISION AVOIDANCE: WARNINGS

전체 16,964건

- └ 80.5% (13,660건) - 애초에 1단계뿐 → 2단계 제한과 무관, 손실 없음
- └ 18.2% (약 3,083건) - 정확히 2단계 → 그대로 다 보존됨
- └ 1.3% (221건) - 3단계 이상 → 2단계로 자르면 이 부분만 정보 손실

즉 98.7%는 2단계 제한으로 아무 손실이 없고, 손실이 생기는 건 딱 1.3%(221건)뿐이에요. 그 221건도 확인해보면 대부분 ELECTRICAL SYSTEM:PROPULSION SYSTEM:TRACTION BATTERY:MANAGEMENT SYSTEM/ENERGY CONTROL MODULE (BMS/BECM) 처럼 4단계까지 파고든 극히 세부적인 항목들이라, 우리 프로젝트의 8종 part_category 체계에서는 어차피 크게 안 중요한 세밀도예요.

3. Unknown or other는 어떻게 처리할 것인가?

16900건 중에 33%가 차지. 근데 아래와 같이 쪼개짐.

→ 형제행이 있는 unknown에 대해서는 그 형제행을 제외한 다른 내용을 보게 설계

→ 형제행이 없는 유일한 unknown은 기존에 5~9 하는 방식으로 채택

→ 이미 raw data에서 분류가 완료된 컬럼에서 제미나이가 또 판단하게 하는 것보다는 그대로 유지하고 위의 방식이 더 낫다고 판단

확인해보니 33.7%가 다 같은 문제는 아니고, 두 종류로 쪼개져요.

실제 데이터로 나눠보면

유형	건수	비율
UNKNOWN OR OTHER가 그 ODINO의 유일한 행 (다른 형제 행 없음)	4,509건	79.0%
같은 ODINO에 다른 구체적 COMPDESC 행도 같이 있음	1,202건	21.0%

▼ 구조화 완료 변수명

#	필드	방식
1	cmplid	원본 CMPLID 그대로 (코드, LLM 관여 없음)
2	odino	원본 ODINO 그대로 (코드, LLM 관여 없음)
3	compdesc1	원본 COMPDESC 를 : 로 잘라 첫 조각(대분류) (코드, LLM 관여 없음)
4	compdesc2	두 번째 조각(중분류). 없으면 "no" 로 채움. 3번째 조각 이상은 버림 (코드, LLM 관여 없음)
5	symptoms	Gemini 판단, compdesc1/2 앵커링, 최대 5개 (3개에서 상향)
6	severity	Gemini 판단, compdesc1/2 앵커링
7	driving_context	Gemini 판단, compdesc1/2 앵커링
8	evidence_quote	Gemini 판단, compdesc1/2 앵커링
9	insufficient_info	Gemini 판단, compdesc1/2 앵커링

▼ compdesc 분류

hk_electrical_recent_full.csv 기준

1. 대분류(compdesc1)

compdesc1	건수	비율	설명
ELECTRICAL SYSTEM	6,837	40.3%	(실측 기반) 차량의 일반 전기·전자 계통 전반을 아우르는 포괄 범주 — 실제 중분류를 보면 점화장치(IGNITION)·경적(HORN)·계기판(INSTRUMENT PANEL/CLUSTER)·배터리(12V/24V/48V)·알터네이터·배선(WIRING)·스타터·BCM(차체 제어모듈)·소프트웨어 등 ADAS가 아닌 "차량 전반의 전기 관련 결함"을 모두 여기로 분류함
UNKNOWN OR OTHER	5,711	33.7%	(공식, CLAUDE.md에 이미 기록) NHTSA가 접수 시점에 결함 부위를 특정하지 못했거나 분류 체계에 없는 경우 붙이는 라벨. 프로젝트에서 이미 확인한 대로 부정확할 수 있는 라벨이며, 실제로는 심각한 결함이 이 라벨로 잘못 분류된 사례가 확인됨 — 그래서 v4에서 형제 COMPDESC 조회 로직까지 별도로 만든 것
FORWARD COLLISION AVOIDANCE	2,057	12.1%	NHTSA 공식: 전방 충돌 회피 시스템 — 센서로 저속·정지 차량을 감지해 충돌이 임박하면 경고를 주거나(전방충돌경고), 자동으로 제동(자동긴급제동/AEB)해 충돌을 막는 기능
VEHICLE SPEED CONTROL	1,439	8.5%	(실측 기반) 차량 속도를 운전자 의도와 무관하게 제어하는 계통 — 중분류를 보면 크루즈 컨트롤(CRUISE CONTROL)·가속 페달(ACCELERATOR PEDAL)·스로틀(THROTTLE) 등, 의도치 않은 가속/감속·크루즈 오작동 관련 결함이 여기 분류됨
LANE DEPARTURE	630	3.7%	NHTSA 공식: 차선이탈경고 시스템 — 카메라로 차선 표시를 인식해 차량이 차선을 벗어나려 하면 운전자에게 음향·시각 등으로 경고
BACK OVER PREVENTION	273	1.6%	NHTSA 공식: 후진 시 후방 자동제동 — 후방 센서·후방카메라로 후진 중 물체를 감지해 충돌이 임박하면 자동으로 제동
ELECTRONIC STABILITY CONTROL (ESC)	17	0.1%	NHTSA 공식: 전자식 차체자세제어장치 — 차량이 방향 안정성을 잃기 시작할 때(후륜 스핀아웃·전륜 언더스티어) 바퀴별로 자동 제동을 걸어 운전자의 차량 제어를 돕는 시스템

2. 중분류(compdesc2)

compdesc1 (대분류)	compdesc2 (중분류)	건수
ELECTRICAL SYSTEM (총 6,837건)	no	6,470
	IGNITION	70
	HORN	62
	INSTRUMENT PANEL	59
	INSTRUMENT CLUSTER/PANEL	49
	12V/24V/48V BATTERY	42
	PROPULSION SYSTEM	21
	ALTERNATOR/GENERATOR/REGULATOR	16
	ADAS	15
	WIRING	13
	STARTER ASSEMBLY	11
	BODY CONTROL MODULE/BCM	2
	SEAT HEATER	2

compdesc1 (대분류)	compdesc2 (중분류)	건수
	SOFTWARE	2
	HAND HEATER	1
	CYBERSECURITY	1
	INTEGRATED TRAILER BRAKE CONTROL	1
UNKNOWN OR OTHER (총 5,711건)	no	5,711
FORWARD COLLISION AVOIDANCE (총 2,057건)	AUTOMATIC EMERGENCY BRAKING	822
	WARNINGS	742
	ADAPTIVE CRUISE CONTROL	435
	no	51
	SENSING SYSTEM	4
	DYNAMIC BRAKE SUPPORT/BRAKE ASSIST	2
	SOFTWARE	1
VEHICLE SPEED CONTROL (총 1,439건)	no	1,375
	ACCELERATOR PEDAL	42
	CRUISE CONTROL	12
	THROTTLE	8
	CABLES	1
	SPRINGS	1
LANE DEPARTURE (총 630건)	BLIND SPOT DETECTION	245
	ASSIST	221
	WARNING	143
	LANE KEEP	13
	no	6
	SENSING SYSTEM	2
BACK OVER PREVENTION (총 273건)	WARNINGS	124
	REARVIEW SYSTEM BRAKING	90
	no	31
	SENSING SYSTEM	23
	AUTOMATIC SYSTEM BRAKING	3
	DISPLAY FUNCTION	2
ELECTRONIC STABILITY CONTROL (ESC) (총 17건)	no	16
	CONTROL MODULE	1

배경 — v4로 바뀐 것

원본 NHTSA 데이터(hk_electrical_recent_full.csv, 16,964행)를 다시 들여다본 결과, 같은 신고(ODINO)가 여러 결함 유형을 가질 때 NHTSA가 이미 CMPLID(행) 단위로 분리해줬다는 걸 발견했다. v4는 이 구조를 그대로 활용해 처리 단위를 ODINO → CMPLID로 바꾸고, COMPDESC (부품 분류)를 코드가 compdesc1 (대분류)/ compdesc2 (중분류)로 잘라 LLM에게 "이 앵커에만 집중하라"는 입력으로 준다. compdesc1 이 UNKNOWN OR OTHER 일 때만 같은 ODINO의 다른 행(형제) COMPDESC를 조회해 "이미 처리된 부분은 배제하라"는 근거로 추가 제공한다. 자세한 설계는 struct_prompt_v4.md 참조.

표본 — sample_100_v4.csv

hk_electrical_recent_full.csv(16,964행, 51개 원본 컬럼 전부)에서 100건 추출(sample_v4_100.py) → 결과물: sample_100_v4.csv

그룹	건수	목적
A_multi_cmplid	27행 (11개 ODINO)	다중 CMPLID 앵커링이 실제로 작동하는지 검증 — ODINO의 행 전부 포함
B_unknown_with_sibling	13행	UNKNOWN OR OTHER + 형제 COMPDESC 있음 케이스 검증
C_unknown_no_sibling	13행	UNKNOWN OR OTHER + 단독 행 케이스 검증
RANDOM	47행	나머지 전체에서 무작위
합계	100행	고유 ODINO 84건, 고유 CMPLID 100건

실행

- LLM: Gemini 2.5 Flash, temperature=0(운영 기본값), `responseMimeType=application/json`
- 형제 COMPDESC 조치는 표본이 아니라 **원본 전체(hk_electrical_recent_full.csv)**를 인덱스로 사용 — 표본 밖에 있는 형제 행도 정확히 찾아냄(검증 완료)
- 실행 스크립트: `struct_verify_v4.py`

```
python scripts/str01_batch_structurize.py \  
--input data/samples/sample_100_v4.csv \  
--prompt docs/struct_prompt_v4.md \  
--output data/processed/str01_sample100_v4_results.jsonl \  
--sibling-source data/processed/hk_electrical_recent_full.csv
```

결과 파일: `str01_sample100_v4_results.jsonl` / 리포트 전체: `str01_sample100_v4_report.md`

STR-02 — 인용(환각) 자동 검사✓

이전 버전(str01_sample100_v4_report.md)과 달라진 점

1. **프롬프트**: `docs/struct_prompt_v4.md` 에서 "서술문에 이 COMPDESC 관련 내용이 없는 경우"(진짜 정보부족 예시, 구 예시4)를 삭제 — 이 예시가 오히려 모델의 정보부족 판정을 과도하게 유도하는 부작용이 있어 제거. 나머지 예시 번호를 1~5로 재정렬(기존 6개 → 5개).
2. **코드**: `scripts/str01_batch_structurize.py` 에 후처리 추가 — LLM 응답의 `symptoms` / `evidence_quote` 가 비어있으면 코드가 강제로 안내문구(["정보 부족으로 판단 불가"] / "(정보 부족으로 근거 문장 없음)")로 채움. **어떤 경우에도 빈 배열·빈 문자열이 최종 출력에 남지 않도록 보장.**
3. **검증 스크립트 버그 수정**: `scripts/struct_verify_v4.py` 가 원본 CDESCR의 모지바케(UTF-8 → latin-1 오디코딩)를 복구하지 않고 비교해 정상 인용을 환각으로 오판하던 버그 발견·수정(Task 12에서 이미 한 번 발견됐던 동일 클래스 버그가 재발한 것 — `fix_mojibake()` 호출 추가).

실행

동일 100건(`data/samples/sample_100_v4.csv`)을 동일 방식(temperature=0, Gemini 2.5 Flash)으로 재실행:

```
python scripts/str01_batch_structurize.py \  
--input data/samples/sample_100_v4.csv \  
--prompt docs/struct_prompt_v4.md \  
--output data/processed/str01_sample100_v4_results.jsonl \  
--sibling-source data/processed/hk_electrical_recent_full.csv
```

100/100 완주, 실패 0건.

항목	결과
스키마 위반	0/100
compdesc1/compdesc2 불일치(코드 필드 자체 검증)	0/100
인용 불일치(환각)	0/100
<code>insufficient_info=true</code> + 안내문구 예외 허용	14/100

인용 통과율 100%(불일치 0/100) . `insufficient_info=true` 인 14건은 **전부 `evidence_quote="(정보 부족으로 근거 문장 없음)"` **였고(빈 문자열 아님), `symptoms` 도 전부 `["정보 부족으로 판단 불가"]` (빈 배열 아님) — 이전 버전과 달리 셀 자체가 비어있는 경우가 하나도 없다.

`compdesc1` / `compdesc2` 는 LLM 출력이 아니라 코드가 원본 COMPDESC를 그대로 파싱해 채우는 필드라, 100건 전부 원본과 정확히 일치하는지도 별도로 재확인(불일치 0건).

- severity: CRITICAL 61 / SERIOUS 23 / MINOR 9 / MODERATE 7

분포 (참고 — 정답 아님, STR-03에서 안정성 진단)

- driving_context: 주행 중 57 / 불명 28 / 정차·주차 중 13 / 시동 시 2
- compdesc1: UNKNOWN OR OTHER 45 / ELECTRICAL SYSTEM 23 / FORWARD COLLISION AVOIDANCE 15 / VEHICLE SPEED CONTROL 9 / LANE DEPARTURE 5 / BACK OVER PREVENTION 3
- insufficient_info=true: 14건
- symptom 태그: 총 302개(고유 258개)
- sample_group: RANDOM 47 / A_multi_cmplid 27 / B_unknown_with_sibling 13 / C_unknown_no_sibling 13

STR-03 — 확장 정확도 측정

개요

- 대상: 100건 (동일 100건을 v4 프롬프트, temperature=0.4로 3회 독립 실행, CMPLID 매칭)
- 비교 대상 정형필드 3개: severity·driving_context·insufficient_info (v3의 part_category는 v4엔 없음 =compdesc1/compdesc2로 대체, 코드가 채우는 결정론적 필드라 비교 불필요. mentions_existing_recall도 v4 스키마엔 없음)
- 목적: 다른 LLM·사람 라벨과 비교하는 게 아니라, 같은 모델이 스스로 얼마나 일관되게 판단하는지 측정 — 흔들리는 레코드 = 사람 검토 우선순위
- 산출물: `data/processed/str03_consistency_v4_fixed_detail.csv` (전 건 상세), 이 리포트

필드별 3회 일치율 (정형값 3개)

필드	3/3 일치(안정)	2/1 (부분흔들림)	1/1/1 (완전분열)
심각도 (severity)	77 (77.0%)	19 (19.0%)	4 (4.0%)
주행 상황 (driving_context)	82 (82.0%)	16 (16.0%)	2 (2.0%)
정보부족 여부 (insufficient_info)	90 (90.0%)	10 (10.0%)	0 (0.0%)

레코드별 불안정 점수 분포 (3개 필드 중 일치하지 않는 필드 수)

불안정 점수	건수
0	68
1	20
2	5
3	7

등급	건수	비율
안정	68	68.0%
부분 흔들림	20	20.0%
불안정	12	12.0%

표본 그룹별 불안정 비율 (v4 고유 메커니즘 검증)

그룹	안정	부분 흔들림	불안정	총	불안정+부분흔들림 비율
A_multi_cmplid	20	4	3	27	25.9%
B_unknown_with_sibling	6	4	3	13	53.8%
C_unknown_no_sibling	11	2	0	13	15.4%
RANDOM	31	10	6	47	34.0%

A_multi_cmplid(다중 CMPLID 앵커링)·B_unknown_with_sibling(형제 COMPDESC 조회) 그룹의 불안정 비율이 RANDOM보다 뚜렷이 높다면, 이 두 메커니즘 자체가 판단 흔들림을 더 유발한다는 뜻으로 해석 가능 — 다만 각 그룹 표본이 13~27건으로 작아 확정적 결론은 아니다.

보조지표 — evidence_quote 위치 겹침(LoU) / symptoms 단어 자카드

- 안정 레코드(3필드 전부 일치, n=68) 평균 evidence 겹침(LoU): 0.803
- 불안정 레코드(1개 이상 필드 흔들림, n=32) 평균 evidence 겹침(LoU): 0.758
- 안정 레코드 평균 symptom 자카드: 0.577
- 불안정 레코드 평균 symptom 자카드: 0.460
- severity 3회 불일치 레코드(n=23) 평균 evidence 겹침(LoU): 0.822
- severity 3회 일치 레코드(n=77) 평균 evidence 겹침(LoU): 0.782

해석 가이드: evidence 겹침이 높음에도 판단이 같았다면 '같은 근거를 보고도 규칙 적용이 흔들린 것'(프롬프트 규칙이 모호할 가능성), 겹침이 낮다면 '애초에 다른 단서를 근거로 삼은 것'(원문 자체가 여러 단서를 담고 있어 모델이 어느 것을 볼지부터 흔들린 것)으로 구분해서 읽는다.

temperature와 결과의 관계 — 해석상 반드시 짚어야 할 점

이 실험에서 관측된 흔들림은 **전부 temperature=0.4로 일부러 올린 결과다**. 이걸 숨은 결함이 아니라 self-consistency 진단 방법 자체의 설계 원리 — "살짝 흔들어서 어느 판단이 약한 다리인지 본다"는 것이라, 흔들림이 온 도 때문에 발생하는 것 자체는 의도된 동작이다. 다만 이로 인해 이 데이터로 정당하게 주장할 수 있는 것과 없는 것이 갈린다.

- **정당한 주장 — 상대적 순위**: T=0.4에서도 3필드 전부 그대로인 레코드는 모델이 강하게 확신하는 케이스다. 반대로 흔들린 레코드는 온도를 얼마로 잡든 상대적으로 더 흔들릴 가능성이 높은 후보군 — 이 순위를 "사람이 먼저 봐야 할 우선순위"로 쓰는 것은 타당하다.

- **부당한 주장 — 절대 수치:** 위 필드별 일치율을 "Gemini 운영(temperature=0) 판정이 이 정도로 불안정하다"고 읽으면 안 된다. T를 더 올렸으면 이 숫자는 더 나빠졌을 것이고, 이는 모델의 실제 신뢰도가 아니라 이번에 고른 T=0.4라는 진단 강도를 반영하는 수치다. 이 실험은 T=0(운영값) 3회 반복이라는 대조군 없이 진행됐다.
- **표본 그룹별 비교도 참고 가설:** 위 '표본 그룹별 불안정 비율'은 각 그룹 13~27건 표본에서 나온 관찰이라, 확정된 결론이 아니라 추가 검증이 필요한 가설로 취급한다.
- **결론:** 이번 실험의 확실한 산출물은 "이 100건 중 어디를 먼저 사람이 봐야 하는가"라는 우선순위 리스트(상세 CSV)이고, 절대 수치나 그룹 간 비교 해석은 참고 가설로만 취급한다.

STR-04 — 스키마 v3 (mentions_existing_recall)

STA-01,02 정윤님 파트에서 아래와 같은 방식으로 구현되기 때문에 굳이 추가할 필요 없는 변수라고 판단

▼ STA-01,02 대략적 구현 절차

리콜DB 데이터를 기반으로, 이 차종×부품 조합에 진짜 리콜 이력이 있는지 먼저 대조->

있으면 신고 날짜들 사이 공백이 6개월 넘었다가 다시 신고 들어오면 재발, 6개월째 신고 없으면 잠잠, 아니면 그냥 리콜로 분류

그리고 나서 리콜 이력 없는 조합은 최근 신고 급증 여부로 증가/신규 나눔